Exercise 1: Order Book Manager

Problem

Design and implement an order book manager that reads a sequence of order and trade messages, handle error conditions and disseminate price information from the constructed order book

Background

Electronic exchanges typically publish a market feed to disseminate market events with data such as buy and sell prices, last trading price. Every time a participant inserts an order, an order is modified or cancelled, or two orders match to produce a trade, a message is published.

Exchange participants use feed handlers to build order books and relay trade events. A feed handler receives messages, parses them, then updates one order book per exchange product.

We assume the following rules:

- when an order crosses the opposite limit (e.g. if we receive Buy @ 10 on an existing best Sell @ 10 or 9), a trade message is expected,
- when receiving a trade message, the feed handler must remove order quantity starting from the best price (greatest for buys, lowest for sells) and for a given price from the first-received order.

The purpose of this exercise is to build a feed handler that manages order books.

Messages

The types of messages that can arrive on the exchange feed are as follows:

Order: productid, action, orderid, side, quantity, price (e.g., N,388,123,B,9,1000)

productid = unique product identifier on the exchange, positive integer
action = N (new), R (remove), M (modify)
orderid = unique positive integer to identify each order;
used to reference existing orders for remove/modify
side = B (buy), S (sell)
quantity = positive integer indicating maximum quantity to buy/sell
price = double indicating max price at which to buy/min price to sell

Trade: action, quantity, price (e.g., X,388,2,1025)

action = X (trade)
productid = unique product identifier on the exchange, positive integer
quantity = amount that traded
price = price at which the trade happened

Example

This is an example of message sequence that the feed handler has to deal with.

Note that order 100004 is canceled, so it doesn't appear in the book above.

N,5,100000,S,1,1075
N,5,100001,B,9,1000
N,5,100002,B,30,975
N,5,100003,S,10,1050
N,5,100004,B,10,950
N,5,100005,S,2,1025
N,5,100006,B,1,1000
R,100004,B,10,950 // cancel
N,5,100007,S,5,1025

The following sequence of additional messages represents the new order and subsequent trades, removals, and modifies:

N,5,100008,B,3,1050
X,5,2,1025
X,5,1,1025
R,100008,B,3,1050
R,100005,S,2,1025
M,100007,S,4,1025 // order is modified down to reflect quantity traded

Requirements

(0) The program should be able to read a sequence of messages in a text files. Create your C++ program from scratch, using boost or C++11 if possible.
(1) Given a sequence of messages, as defined above, construct an in-memory representation of the current state of the order book. You will need to design your own dataset to test your code.
(2) Every 10^{th} message and on exit, write out a human-readable representation of the book down to the 5^{th} level for each product id.
(3) Write out the total quantity traded at the most recent trade price on every trade message. For example
X,5,2,1025 => product 5: 2@1025

X,5,1,1025 => product 5: 3@1025
X,5,1,1000 => product 5: 1@1000 // quantity resets when new trade price is seen
X,5,1,1025 => product 5: 1@1025 // doesn't remember the previous trades @1025
(4) If your program gets the following garbage input, make sure it collects errors and prints a summary on exit.
a) Corrupted messages
b) Duplicated order ids (duplicate adds)
c) Trades with no corresponding order
d) Removes with no corresponding order
e) Best sell order price at or below best buy order price but no trades occur
f) Negative, missing, or out-of-bounds prices, quantities, order ids
g) Negative product id

Exercise 2: Matching Engine

Problem

You are required to implement a matching engine as part of the exchange simulator in a back testing platform.

Background

Order matching engines are one of the core components in modern exchanges. This component accepts buy and sell orders from clients and matches them up according to various algorithms. One algorithm adopted by most exchanges is price/time priority, which means orders with better price would get matched first and orders at the same price are matched according to the sequence they enter the matching engine.

Requirements

- The matching engine will accept a stream of market data to simulate the market order book. Each update will be a 5-level snapshot as shown below

	Bid5	Bid4	Bid3	Bid2	Bid1	Ask1	Ask2	Ask3	Ask4	Ask5
Price	99.6	99.7	99.8	99.9	100	100.1	100.2	100.3	100.4	100.5
Quantity	500	400	300	300	100	500	1000	2000	2500	3000

- Three basic types of transactions should be supported: insert, amend and cancel.
- When the matching engine receives a transaction, it will validate the transaction first and send back an Ack message that indicates if the transaction is valid.
- When order is filled/partial filled, the matching engine will send a trade message to the client. Each trade message should only have one trade price and trade quantity.
- When there is a change in order state, fill/partial fill/modified/cancelled/completed, the matching engine should send an order status message to the client
- Two matching modes need to be supported and can be easily switched
    - PERCENT. Order would be filled immediately with X% probability and Y% of order size regardless of the current state of market order book. Trade price can be order price or the best price in the market. Orders not matched will be closed and won't remain in the market. X and Y should be configurable.

    - MARKET. Price/Time priority algorithm is used. Please try your best to simulate the real market matching engine behavior.
- It should be easy to add new modes to your program.
- You need to design your own data structure to represent the Order/Ack/Trade messages. The solution should be expandable and have a clean interface.
- Please optimize for the best performance. The performance will be measured in terms of number of transactions per second.

Exercise 3: Basket Pricer

Problem

Design and implement a real-time basket pricer component where a basket can be any set of instruments that are tradable and have a market price. Each instrument has a weight associated with it that is fixed for the day but the market prices are updated real-time

Background

A basket pricer is an essential component to implement various arbitrage strategies to calculate the fair price of a tradable instrument like ETF or Index Future. Depending on the strategy the basket and its related instrument are traded to capture the arbitrage opportunity. Some of the strategies can be extremely latency sensitive and opportunity might exist only for a few micro seconds.

Requirements

1. Read baskets, instruments and weights from a csv file with the below format

Basket	Constituent	Shares
b1	c1	322
b1	c2	323
b2	c1	412
b2	c3	223
b2	c4	100

*Basket price is a sum product of constituent price and shares

2. Call a subscribe method that takes a function object and a list of instruments as arguments

3. Function object that is passed to the subscribe method is called back for each instrument with tick data update on bid, ask and last traded price

4. Basket mid and last need to be recomputed for every tick data update on any of the constituent

5. When the change of basket mid or last is more than the defined threshold the results are printed to standard output

Exercise 4: Restaurant Throughput Management

Problem

Design and implement a generic mechanism to limit the number of meal requests being sent to the central kitchen each second. A solution which can be reused for unrelated but similar purposes is preferred.

Background

The company's central kitchen accepts electronic meal orders from employees via the intranet. Employees can order a meal, change the order, or cancel it outright. Each of these actions requires a message to be sent to the kitchen computer system.

The kitchen has a limited number of human chefs and can only process a limited number of messages each second. A rule has been instituted that each terminal will be allowed to send 3 food requests per second (FPS). Any messages in excess of the limit will be rejected. If too many rejections are generated the session may be terminated and the chefs might ban the employee from the company restaurant. The exact figure of 3/sec may change in the future.

Currently messages tend to be sent in short but intense bursts. A solution is needed to slightly delay messages in excess of the limit to prevent violating kitchen rules without rejecting messages.

It is also important to treat some messages as more urgent than others. For instance, cancel messages should be carried out before new order requests to prevent food from being wasted and to allow the chefs to work on active requests.

You may want to use this same implementation elsewhere in your company (such as a message throttle between two busy components), so a generic reusable design would be a great benefit.

Requirements

1. The solution shall use a sliding window calculation for FPS (food per second). This means that at any given point in time, the amount of transactions sent in the last 1000 milliseconds must be considered. This is opposed to a fixed window method where the count is reset on seconds boundaries based on the wall clock.
2. Messages in excess of the given limit (3/sec) must be queued
    a. The solution must delay only for the minimal amount of time required to be inside the limit. For example, if 1 message was sent every 10 milliseconds for the last 30 milliseconds, the next order should be delayed no longer than 70ms (you can assume the machine has a very precise clock).
    b. Messages inside the limit should be sent immediately with minimum delay

3. Any structure can be used for messages – this is left to the imagination of the implementer and is not considered relevant to the problem. Use whatever makes the solution clearer or cleaner
4. When queued, cancel messages must be sent before other messages similarly queued
    a. Messages should otherwise be sent in the same order they were queued
5. When some arbitrarily large number of messages have already been queued, the solution may start outright rejecting additional messages
6. Optimize for the best case scenario – the majority of messages are not expected to queue